

MEETHI KAHANI

"Every Cookie Tells A Sweet Story"

INDIVIDUAL DEVELOPER SPECIFICATION: DEVELOPER 2

Assigned Developer:	Developer 2
Assigned Role:	Product Catalog & Bakery Kitchen Operations Specialist
Assigned Workspaces:	src/modules/ProductCatalog/ & src/modules/SellerPortal/
Primary Tech Stack:	React 18 + JavaScript + Vite + Pure Vanilla CSS3
AI Prompting Strategy:	2 Focused Component Prompts (Zero Hallucination Guarantee)

1. Workspace Scope & Assigned Modules

You are strictly assigned to **src/modules/ProductCatalog/** & **src/modules/SellerPortal/**. Do NOT edit files outside your assigned folders.

- ProductCatalog Module: Category Filter Tabs, Cookie Cards, Dedicated PDP (/product/:slug).
- SellerPortal Module: Secret Bakery Kitchen (/seller), Live Kitchen Queue, Stock Restock (+50 Baked), Packaging Inventory Tracker.

2. Shared Data Contract (`StoreContext.jsx`)

All modules consume state from `StoreContext.jsx`. Import the hook in your components:

```
import { useStore } from '../../context/StoreContext';
```

Variables available to your module: cookies, categories, activeCategory, setActiveCategory, addToCart, orders, updateOrderStatus, restockCookie, packagingStock, updatePackagingStock

3. Focused AI Prompt #1 — ProductCatalog & PDP Module (src/modules/ProductCatalog/)

Copy and paste this prompt when building **ProductCatalog & PDP Module (src/modules/ProductCatalog/)** to prevent AI confusion:

You are an expert React 18 & Vanilla CSS frontend engineer building ProductCatalog for Meethi Kahani.

STRICT CONSTRAINTS:

1. ONLY write code inside 'src/modules/ProductCatalog/ProductCatalog.jsx', 'ProductCard.jsx', 'ProductDetailsPage.jsx', and 'ProductCatalog.css'.
2. Consume shared state: import { useStore } from '../..context/StoreContext';
3. Access context: cookies, categories, activeCategory, setActiveCategory, addToCart.

DESIGN TOKENS (Vanilla CSS):

- Crimson Maroon: #9A2222 | Vanilla Linen: #FAF5EE | Dark Cocoa: #3D2314 | Warm Gold: #D4A359

KEY FEATURES TO BUILD:

- Category Filter Tabs (Cookies, Gift Sets, Festive Combos).
- Cookie Product Cards displaying price (■), pieces count, native images, out-of-stock badges, and Quick Add button.
- Dedicated Product Details Page (/product/:slug): Left side 3-4 thumbnail multi-angle photo gallery; Right side Title, Price, Quantity Selector, Add to Cart, Pincode Checker, and "The Kahani Behind This Cookie" story card at the end.

Generate clean, production-ready React code with CSS scoped inside ProductCatalog.css.

4. Focused AI Prompt #2 — SellerPortal Kitchen Module (src/modules/SellerPortal/)

Copy and paste this prompt when building **SellerPortal Kitchen Module (src/modules/SellerPortal/)**:

You are an expert React 18 & Vanilla CSS frontend engineer building the Bakery Kitchen Seller Portal for Meethi Kahani.

STRICT CONSTRAINTS:

1. ONLY write code inside 'src/modules/SellerPortal/SellerDashboard.jsx', 'KitchenQueue.jsx', 'PackagingManager.jsx', and 'SellerPortal.css'.
2. Consume shared state: import { useStore } from '../..context/StoreContext';
3. Access context: orders, updateOrderStatus, cookies, restockCookie, packagingStock, updatePackagingStock.

DESIGN TOKENS (Vanilla CSS):

- Crimson Maroon: #9A2222 | Vanilla Linen: #FAF5EE | Dark Cocoa: #3D2314 | Warm Gold: #D4A359

KEY FEATURES TO BUILD:

- Secret Bakery Kitchen Portal (/seller) with login credentials protection.
- Live Kitchen Order Fulfillment Queue (Received -> Baking -> Shipped -> Delivered).
- Stock Restock Manager ("+50 Baked" quick increment button).
- Packaging Material Inventory Manager (tracks Box Tins, Wax Seals, Satin Ribbons; auto-decrements stock when orders are marked packed).

Generate clean, production-ready React code with CSS scoped inside SellerPortal.css.

5. Golden Rules for Zero-Error Integration

- **Rule 1:** Execute Prompt #1 first, test locally, then execute Prompt #2.
- **Rule 2:** Never edit code in another teammate's module directory.
- **Rule 3:** Keep CSS classes scoped inside your module's CSS file.

MEETHI KAHANI FOODS PVT LTD — TEAM MASTER BLUEPRINT